

Concept-Guided Spatial Regularization for World Models in Atari Pong

Yukuan Lu
UC Davis

Zaishuo Xia
UC Davis

Weyl Lu
UC Davis

Yubei Chen
UC Davis

Abstract

World models are usually evaluated as components of model-based reinforcement learning (MBRL) systems, while the world models themselves are rarely studied in isolation.

We examine five representative visual world-model agents in Atari Pong—DreamerV3, DIAMOND, TWISTER, Stimulus, and STORM. After reproducing their training pipelines and matching the reported agent performance, we freeze the learned world models and evaluate them with a closed-loop rollout diagnostic: a policy trained separately from the corresponding MBRL agent interacts with each frozen model, and the generated video trajectories are inspected for visual and dynamical errors. Across all five models, the rollouts contain clear failures, including ball disappearance, incorrect ball motion, and invalid ball–paddle interactions.

Beyond visual trajectories, we further evaluate them with pixel-space zero-shot MBRL, where a new policy is trained entirely inside a frozen world model and then evaluated in the real environment. Across all five models, the resulting policies substantially underperform those produced by the corresponding original MBRL training pipelines. The gap is particularly large for DreamerV3, whose mean return drops from -5.5 to -20.9 , near the minimum Pong return of -21 .

We hypothesize that insufficient modeling of task-critical concepts, such as the ball in Pong, may contribute to these failures. We therefore propose Concept-Guided Spatial Regularization (CGSReg), an auxiliary pixel reconstruction loss applied to segmented concept regions. Experiments show that CGSReg improves both closed-loop rollouts and pixel-space zero-shot MBRL in DreamerV3, DIAMOND and TWISTER. Its effects vary across the remaining models and evaluation metrics, indicating that CGSReg alone does not address all world-model bottlenecks.

1 Introduction

World models are commonly evaluated through the reinforcement-learning performance of the complete model-based reinforcement learning (MBRL) agent. However, strong agent performance does not guarantee that the learned world model is a reliable simulator on its own. In Atari Pong [Bellemare et al., 2013], we reproduce five representative visual world-model agents: DreamerV3, DIAMOND, TWISTER, Stimulus, and STORM [Hafner et al., 2023, Alonso et al., 2024, Burchi and Timofte, 2025, Cohen et al., 2025, Zhang et al., 2023]. Their agent performance matches the original reports, yet their frozen world models are unreliable when evaluated on their own. Their generated trajectories contain failures such as ball disappearance and incorrect rebounds, and policies trained from scratch inside these models transfer poorly to the real environment. The contrast is especially large for DreamerV3: its mean Pong return falls from -5.5 for the original agent to -20.9 when policy learning is performed inside the frozen world model, near the minimum return of -21 .

Many of these rollout failures involve the ball and its interactions, despite the ball occupying only a few pixels. This leads us to investigate whether task-critical concepts receive insufficient learning signal. We propose Concept-Guided Spatial Regularization (CGSReg), which adds reconstruction

Algorithm 1 Dyna-style model-based reinforcement learning

```
1: Initialize policy or planner  $\pi$ , world model  $M_\phi$ , and replay buffer  $\mathcal{D}$ 
2: for training iteration  $k = 1, 2, \dots$  do
3:   Collect real-environment transitions with  $\pi$  and append them to  $\mathcal{D}$ 
4:   Update  $M_\phi$  on sequences sampled from  $\mathcal{D}$ 
5:   Generate rollouts with  $M_\phi$  under  $\pi$ 
6:   Improve  $\pi$  using the generated rollouts
7: end for
8: return the trained policy or planner  $\pi$  and world model  $M_\phi$ 
```

Table 1: Representative visual world models evaluated in Atari Pong.

World model	Backbone	Short description
DreamerV3	RSSM / recurrent latent dynamics	Predicts latent dynamics and decodes observations.
DIAMOND	Diffusion pixel world model	Models visual dynamics with a pixel-space diffusion model.
TWISTER	Transformer latent world model	Adds action-conditioned representation learning to a STORM-style backbone.
Simulus	Token / RetNet world model	Predicts future discrete visual tokens with sequence modeling.
STORM	Transformer latent world model	Models stochastic latent states with Transformer dynamics.

supervision to image regions corresponding to concepts such as the Pong ball. It improves both closed-loop rollouts and pixel-space zero-shot MBRL in DreamerV3, DIAMOND, and TWISTER, but does not clearly improve Simulus or STORM. This contrast shows that concept-focused reconstruction addresses one important source of world-model error, but not every bottleneck.

A world model predicts future observations from past observations and actions. Depending on the task, an observation may be an image or a state describing the environment. In MBRL, the learned model generates future trajectories in which the policy can be trained.

Many MBRL agents follow a common training pattern: the current policy collects real-environment transitions, the world model is updated on the replay buffer, and the policy is improved using model-generated trajectories. These steps are repeated throughout training, so the replay distribution, world model, and policy evolve together. Following the Dyna framework [Sutton, 1990, 1991], we refer to agents trained in this way as *Dyna-style agents*. Algorithm 1 summarizes this process.

We study five visual world-model projects that follow this Dyna-style training pattern: DreamerV3, DIAMOND, TWISTER, Simulus, and STORM. They span several model backbones, including recurrent state-space models, diffusion models [Ho et al., 2020], Transformers [Vaswani et al., 2017], and RetNet [Sun et al., 2023], as well as different world-modeling paradigms: latent-state prediction, direct pixel-space generation with diffusion models, and autoregressive next-token prediction. Table 1 summarizes these differences.

We focus on Atari Pong, an environment from the Arcade Learning Environment [Bellemare et al., 2013] and the Atari100K benchmark [Kaiser et al., 2020]. Pong contains two vertical paddles and a ball moving across the screen. The agent controls the player paddle on the right, while the game controls the opponent paddle on the left. The agent can move its paddle up or down, remain still, and combine these actions with FIRE, which starts or resumes play. A point is scored whenever one side fails to return the ball. An episode ends when either side reaches 21 points, so the return lies in $[-21, 21]$.

Pong is visually simple, but its dynamics depend on accurately tracking a small ball and modeling its interactions with the paddles and walls. This makes world-model errors easy to inspect and directly relevant to control. In our closed-loop rollouts, the most common failures involve the ball or ball-related events: the ball disappears, changes direction without a valid interaction, or responds incorrectly to paddle collisions.

Pong’s action space contains six actions: NOOP, FIRE, RIGHT, LEFT, RIGHTFIRE, and LEFTFIRE. NOOP leaves the paddle unchanged, while FIRE starts or resumes play. In Pong, RIGHT moves

the player paddle upward and LEFT moves it downward. RIGHTFIRE and LEFTFIRE combine the corresponding movement with FIRE. In our figures, we abbreviate these actions as $-$, F, R, L, RF, and LF, respectively.

We evaluate the frozen world models with two complementary diagnostics. In the *closed-loop rollout diagnostic*, a policy trained separately from the corresponding MBRL agent interacts with the frozen world model, allowing us to inspect the generated video trajectories. Across all five models, these rollouts contain clear failures involving the ball, including disappearance, incorrect motion, and invalid ball-paddle interactions.

Beyond visual trajectories, we introduce *pixel-space zero-shot MBRL*. Zero-shot MBRL was introduced by MoSim [Hao et al., 2025] as a challenging evaluation for state-space world models: a new policy is trained entirely inside a frozen world model and then evaluated in the real environment. We extend this setting to visual world models, where the policy learns directly from generated pixel observations. Across all five models, the resulting policies perform substantially worse than the policies produced by the corresponding original MBRL training pipelines.

Together, these observations motivate our hypothesis that insufficient modeling of task-critical concepts, particularly the ball in Pong, contributes to the frozen-model failures. We therefore propose CGSReg, which adds reconstruction supervision to the corresponding image regions.

To isolate the effect of CGSReg from the co-evolving policy and replay distribution in Dyna-style training, we adopt *offline world-model training*. For each project, we train matched baseline and CGSReg world-model checkpoints on the same fixed replay dataset, with policy learning disabled and the world-model update budget held fixed. CGSReg improves both closed-loop rollouts and pixel-space zero-shot MBRL in DreamerV3, DIAMOND, and TWISTER. Stimulus and STORM show less consistent changes across the two evaluations, revealing bottlenecks not fully addressed by concept-region reconstruction.

Our contributions are:

- We isolate and directly evaluate the frozen world models learned by five reproduced Dyna-style visual MBRL agents. Closed-loop rollouts reveal recurring visual and dynamical failures, while pixel-space zero-shot MBRL shows that these models provide weak standalone environments for policy learning.
- We propose CGSReg, a spatial regularizer that adds reconstruction supervision to task-critical concept regions. In Atari Pong, we instantiate it on the segmented ball region. It improves both closed-loop rollouts and pixel-space zero-shot MBRL in DreamerV3, DIAMOND, and TWISTER.

2 Diagnosing Frozen Visual World Models

We use Pong as a controlled setting for diagnosing frozen visual world models. The visual scene is simple, but the dynamics are sensitive to small objects and precise ball-paddle interactions.

2.1 Closed-Loop Rollout Diagnosis

We drive all five reproduced frozen world models with the same external controller: a pixel-based PPO [Schulman et al., 2017] policy trained with Stable-Baselines3 [Raffin et al., 2021] in the real Atari Pong environment. We use the publicly available PongNoFrameskip-v4 checkpoint from Stable-Baselines3¹. This controller is independent of the policies used to collect the reproduced Dyna-style replay and is only used to probe the frozen simulators. It supplies actions, while the frozen world model predicts the next observation, reward, and termination signal.

Figure 1 shows representative failure cases from the five frozen world models. Although the rollouts often preserve the global Pong scene, they frequently break task-critical objects and interactions. Across these examples, we observe recurring failure modes including ball disappearance, paddle disappearance, spurious ball direction change, and incorrect ball-paddle interaction. These failure modes are not mutually exclusive and can appear across multiple world models. Beyond these object-level failures, TWISTER also exhibits local visual artifacts around the ball. These errors directly affect rewards and policy learning because the ball and paddles determine rebounds and scoring

¹<https://huggingface.co/sb3/ppo-PongNoFrameskip-v4>

events. Furthermore, STORM exhibits an action-response failure, where the generated dynamics do not reliably follow the supplied actions. For example, the generated paddle can return toward its original position without an intervening action that would cause such a movement.

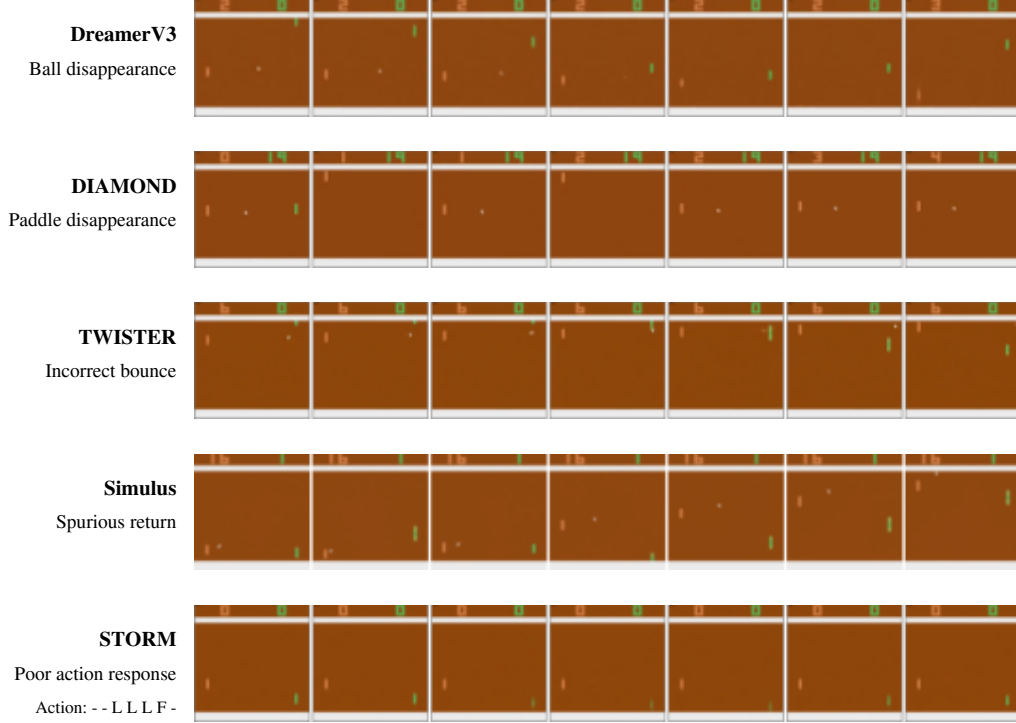


Figure 1: Closed-loop rollout failures across five frozen world models driven by the same external policy. Each row is a manually selected seven-frame window from a longer horizon-512 closed-loop rollout and illustrates one representative failure. In DreamerV3, the ball disappears from $t = 4$ onward. In DIAMOND, the green player paddle on the right is already missing at $t = 1$ and remains absent. In TWISTER, the ball reaches the right player paddle at $t = 4$, overlaps the center of the paddle at $t = 5$, and passes horizontally through it at $t = 6$ instead of rebounding. In Stimulus, the ball strikes the opponent paddle on the left at $t = 1$ and rebounds toward the upper right at $t = 3$ and $t = 4$. From $t = 5$ to $t = 7$, however, it spuriously reverses toward the upper left without colliding with any object. In STORM, the model receives the action sequence $[-, -, L, L, L, F, -]$ from $t = 1$ to $t = 7$, but the player paddle does not respond to the supplied actions. These examples are representative rather than exclusive: the same failure mode may occur in multiple models. The displayed windows are selected only for qualitative illustration.

2.2 Frozen-Model Zero-Shot MBRL Evaluation

Rollout videos reveal visual and dynamical errors, but they do not directly measure whether a frozen world model can support policy learning. We therefore compare two settings for each reproduced project.

First, we evaluate the final policy of the reproduced Dyna-style agent. This policy is obtained from our reproduction of the project’s original joint training pipeline, and its performance matches that reported in the corresponding paper.

Second, we extract and freeze the final world model from the same reproduced agent, discard its policy and value components, and train a new policy from scratch entirely inside the frozen model. We refer to this setting as pixel-space zero-shot MBRL, because the new policy learns from pixel trajectories generated by the frozen world model and is then evaluated in the real environment. We use each project’s native policy-learning procedure, unlike the unified protocol introduced later in subsection 4.3. World-model updates and additional real-environment data collection are disabled throughout policy training.

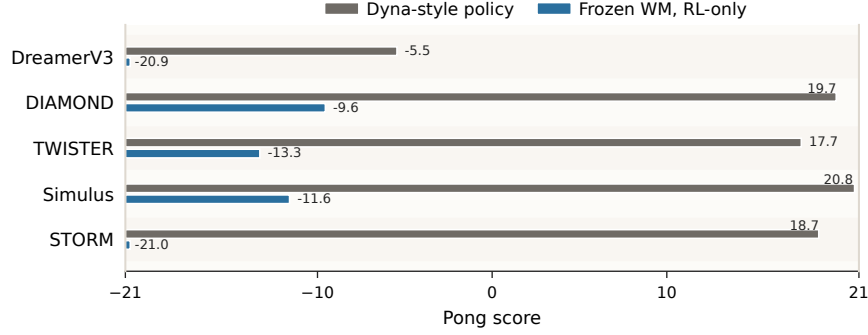


Figure 2: Frozen-model pixel-space zero-shot MBRL diagnostic. For each project, *Dyna-style policy* denotes the final policy of our reproduced Dyna-style agent, while *Frozen WM, RL-only* denotes a newly initialized policy trained inside the frozen world model extracted from that agent. Each bar reports the mean real-environment Pong return; higher is better, and the minimum possible return is -21 . Exact means and standard deviations are provided in Table 2, together with the original-paper scores for the reproduced agents.

As shown in Figure 2, the frozen-WM policies perform far below the corresponding reproduced Dyna-style policies across all five projects. Both policies are evaluated with deterministic action selection over 20 reset seeds in the real ALE Pong environment. Table 2 reports the exact scores and compares our reproduced Dyna-style agents with the scores reported in the original papers.

This diagnostic shows that a world model learned within a strong Dyna-style agent may still be a weak standalone environment for policy learning. Together with the closed-loop rollout failures, this motivates our hypothesis that task-critical concepts, particularly the Pong ball and its interactions, are insufficiently modeled.

3 Concept-Guided Spatial Regularization

CGSReg is an auxiliary regularization term that adds extra image reconstruction supervision to pixels corresponding to important concepts.

For the visual world models studied in this paper, we decompose the original world-model objective into an image-space loss and the remaining nonvisual objectives:

$$L_{\text{wm}} = L_{\text{img}} + L_{\text{nonvisual}}, \quad (1)$$

where L_{img} is the image reconstruction or prediction loss and $L_{\text{nonvisual}}$ collects the remaining objectives, such as dynamics, reward, and continuation losses.

CGSReg adds an additional masked image loss:

$$L_{\text{wm}} = L_{\text{img}} + \lambda_{\text{CGSReg}} L_{\text{CGSReg}} + L_{\text{nonvisual}}, \quad (2)$$

where λ_{CGSReg} controls the regularization strength.

Specifically, we represent the pixels of an important concept with a binary mask m . Given a target image x and a world-model output $\hat{x}$, we define

$$L_{\text{CGSReg}} = \frac{\sum_p m_p (x_p - \hat{x}_p)^2}{\sum_p m_p}, \quad (3)$$

where p indexes individual image values. For RGB images, the mask is applied to all color channels. Normalizing by the mask area ensures that each concept contributes according to its reconstruction error rather than its pixel size.

For Pong, we use the ball as the main task-critical concept and obtain its mask with SAM2 [Ravi et al., 2024]. Paddle masks, multi-mask training, and loss-scale matching are described in Appendix B.

4 Experiments

This section evaluates whether CGSReg improves frozen visual world models. For each project, we compare two matched offline world-model checkpoints and additionally evaluate the reproduced-agent WM, extracted from our reproduction of the original Dyna-style agent.

We adopt offline world-model training introduced in section 1 to isolate the effect of CGSReg from policy learning and replay-buffer evolution in the Dyna-style loop. Under this setting, the baseline and CGSReg checkpoints are trained using the same fixed replay data and matched world-model update budgets, making them the controlled comparison for measuring the effect of CGSReg. Full training details are provided in Appendix D.

The reproduced-agent WM serves a different role. It connects the controlled offline experiments to the frozen world models diagnosed in section 2 and shows how the offline-trained models compare with a world model learned through each project’s native Dyna-style pipeline. It is included only as a reference and is not part of the controlled CGSReg comparison.

After freezing all checkpoints, we evaluate them using the two diagnostics from section 2: closed-loop rollout diagnosis and frozen-model zero-shot MBRL. The rollout evaluation follows the setup in subsection 2.1. For zero-shot MBRL, we use one shared policy-learning protocol across all projects and checkpoints, enabling direct comparison.

4.1 Compared World-Model Checkpoints

For each project, we evaluate three frozen world-model checkpoints:

- **Reproduced-agent WM:** the world model extracted from our reproduction of the original Dyna-style agent.
- **Offline baseline WM:** the $\lambda_{\text{CGSReg}} = 0$ special case of the offline training objective, in which CGSReg is disabled.
- **Offline CGSReg WM:** the corresponding offline world model trained with a selected nonzero λ_{CGSReg} .

The two offline checkpoints form the controlled comparison. They are trained on the same fixed 100k-step Pong replay dataset collected from our DIAMOND reproduction and use the same world-model update budget; their only difference is the value of λ_{CGSReg} .

We use $\lambda_{\text{CGSReg}} = 0.1$ for DreamerV3, 0.01 for DIAMOND, 1.0 for TWISTER, 0.1 for Simulus, and 0.01 for STORM. All main experiments use the ball mask. Weight sweeps are reported in Appendix F.

4.2 Qualitative Closed-Loop Rollouts

We evaluate all frozen checkpoints using the same external controller and horizon-512 rollout setup as in subsection 2.1. We compare the offline baseline and CGSReg rollouts using the same qualitative criteria: object visibility, ball motion, collision behavior, and action response.

Figure 3 shows DreamerV3, for which CGSReg produces the clearest qualitative improvement. The ball remains visible through the displayed interaction and follows a plausible rebound trajectory.

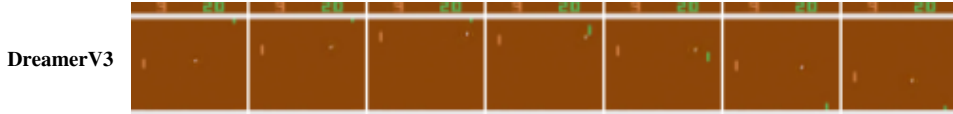


Figure 3: CGSReg closed-loop rollout example for DreamerV3. The frames are sampled from a horizon-512 rollout driven by the same external image-based Pong controller as in Figure 1. The ball starts near the center, moves up-right, rebounds from the player paddle, and continues toward the lower right. The displayed trajectory illustrates stable ball visibility and plausible ball dynamics under CGSReg.

Across the inspected rollouts, DIAMOND and TWISTER also show improved ball modeling and dynamics, although occasional visual artifacts and ball-prediction errors remain. Stimulus does not show a clear rollout improvement, while STORM continues to exhibit the action-response failure identified in subsection 2.1.

4.3 Unified Pixel-Space Zero-Shot MBRL

We apply one shared pixel-space zero-shot MBRL protocol to all three checkpoint types. For each frozen world model, we initialize a new image-based on-policy actor-critic and train it from scratch for 20k updates. As in subsection 2.2, each final policy is evaluated with deterministic action selection over 20 reset seeds in the real ALE Pong environment.

The main comparison is between the matched offline baseline and CGSReg world models. The reproduced-agent WM is included only as a reference. Full protocol details are provided in Appendix E.

The resulting real-environment Pong scores are shown in Figure 4.

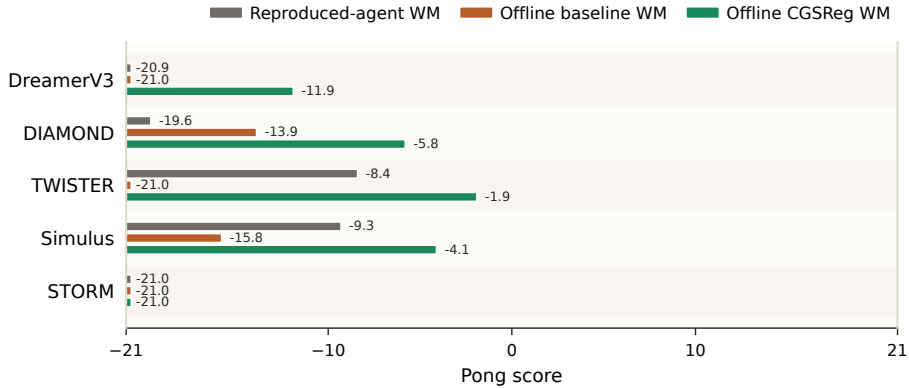


Figure 4: Unified pixel-space zero-shot MBRL scores. Each bar reports the mean real-environment return of a new policy trained from scratch inside the corresponding frozen world model. The reproduced-agent WM bars are not the scores of the original Dyna-style agents. Exact means and standard deviations are provided in Table 3.

Compared with the matched offline baselines, CGSReg raises the mean real-environment Pong return for DreamerV3 ($-21.00 \rightarrow -11.90$), DIAMOND ($-13.90 \rightarrow -5.80$), TWISTER ($-21.00 \rightarrow -1.90$), and Stimulus ($-15.80 \rightarrow -4.10$). STORM remains at -21.00 .

4.4 Architecture-Dependent Outcomes

Stimulus and STORM reveal different limitations of concept-region reconstruction. In Stimulus, CGSReg directly regularizes image-tokenizer reconstruction but not the subsequent token-dynamics model. Its higher zero-shot return therefore does not necessarily indicate better long-horizon token prediction, consistent with the lack of a clear rollout improvement.

For STORM, the main observed failure is its unreliable response to supplied actions. When the generated dynamics do not correctly follow the controller’s actions, improving reconstruction of the ball region cannot address the dominant error.

Overall, CGSReg improves both closed-loop rollouts and pixel-space zero-shot MBRL in DreamerV3, DIAMOND, and TWISTER. The mixed Stimulus result and the unchanged STORM result indicate that concept-focused reconstruction addresses only some world-model bottlenecks.

5 Ablations

The main experiments fix the offline dataset and compare matched $\lambda_{\text{CGSReg}} = 0$ and CGSReg checkpoints. We keep the ablation study separate from this claim. Appendix F reports additional sweeps over λ_{CGSReg} and the offline replay dataset.

6 Limitations and Discussion

CGSReg Alone Is Not Sufficient. Although CGSReg improves pixel-space zero-shot MBRL in several models, the resulting policies still do not solve Pong. As shown in Table 3, their mean returns remain far below the winning return of 21. Their performance also varies substantially across evaluation episodes, particularly for TWISTER and Stimulus, whose standard deviations reach 21.26 and 15.78, respectively. Thus, improving the reconstruction of the ball region is not sufficient to produce a reliable policy-training simulator. Such a simulator also requires long-horizon consistency, accurate reward prediction, reliable action response, and robustness to the policy-induced state distribution.

Evaluation scope. We focus on Atari Pong and five implementations. This setting is useful because the key concept is visible and the failures can be inspected directly. It is also deliberately controlled: we have not systematically varied larger datasets or more complex tasks.

The concepts are manually specified. CGSReg assumes that the important concept is known and can be mapped to a pixel mask. This is a strong human prior. It is reasonable for the Pong ball, but it does not directly cover concepts such as rules, object relations, or latent game states.

A more general approach should discover task-relevant concepts automatically and ground them in their predictive and control consequences. This direction is related to Sutton’s view of world knowledge as empirically grounded predictions [Sutton, 2009] and to Horde, which learns many predictive questions in parallel from sensorimotor experience [Sutton et al., 2011]. Following Sutton’s *The Bitter Lesson* [Sutton, 2019], such a method should rely less on manually specified concepts and benefit from increased data and computation.

Weak frozen models, strong Dyna-style agents. The main unresolved question is why these world models perform poorly as standalone policy-training simulators after freezing, while the corresponding Dyna-style agents can still learn strong Pong policies. Figure 2 shows this gap for the reproduced checkpoints: after freezing the trained world models, we train new policies from scratch inside these frozen models. These policies perform far below the original Dyna-style agents. Figure 4 shows a related problem for the offline-trained checkpoints under the unified zero-shot protocol. Thus, the issue is not tied to one policy-learning protocol: frozen visual world models are poor standalone policy-training simulators, even though Dyna-style MBRL can use them effectively during joint training.

One possible explanation is that continual world-model updating in Dyna-style training lets the policy benefit from a model that is not a strong standalone simulator. In the Dyna-style method, the policy and model are updated together: the policy mainly uses model rollouts near its current behavior, while new real environment transitions are continually added to replay to update the model. In this setting, a world model can improve RL by augmenting the collected real environment data, without becoming a simulator that generalizes well after it is frozen. We tested this by freezing the world model at different points during Dyna-style training and then continuing policy learning (Appendix H). The result is mixed: DreamerV3 and DIAMOND show evidence consistent with this explanation, TWISTER partly supports it, Stimulus does not, and STORM is non-monotonic. Continual world-model updating is therefore important for some architectures, but it does not fully explain why frozen visual world models remain weak policy-training simulators.

7 Conclusion

World models are central to model-based reinforcement learning, but strong agent performance does not guarantee that the learned model is a reliable standalone simulator. We directly evaluate five frozen visual world models from reproduced Dyna-style agents in Atari Pong. Their closed-loop rollouts contain visual and dynamical errors, and policies trained from scratch inside these models perform poorly when transferred to the real environment.

We propose CGSReg to target one possible source of these failures: insufficient modeling of task-critical concepts. Under controlled offline world-model training, CGSReg improves both closed-loop rollouts and pixel-space zero-shot MBRL in DreamerV3, DIAMOND, and TWISTER. Its effects are

less consistent in Stimulus and STORM. These results indicate that concept-focused reconstruction can improve some frozen world models, but does not resolve all world-model bottlenecks.

Future work should generalize CGSReg to broader environments and world-model architectures, and reduce its reliance on manually specified concepts by discovering task-relevant concepts automatically.

References

- Eloi Alonso, Adam Jelley, Vincent Micheli, Anssi Kanervisto, Amos Storkey, Tim Pearce, and François Fleuret. Diffusion for world modeling: Visual details matter in atari. In *Advances in Neural Information Processing Systems*, 2024.
- Marc G. Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47: 253–279, 2013.
- Maxime Burchi and Radu Timofte. Learning transformer-based world models with contrastive predictive coding. In *International Conference on Learning Representations*, 2025.
- Lior Cohen, Kaixin Wang, Bingyi Kang, and Shie Mannor. Improving token-based world models with parallel observation prediction. In *International Conference on Machine Learning*, 2024.
- Lior Cohen, Kaixin Wang, Bingyi Kang, Uri Gadot, and Shie Mannor. Uncovering untapped potential in sample-efficient world model agents. *arXiv preprint arXiv:2502.11537*, 2025.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- Chenjie Hao, Weyl Lu, Yifan Xu, and Yubei Chen. Neural motion simulator: Pushing the limit of world models in reinforcement learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 27608–27617, 2025.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, 2020.
- Lukasz Kaiser, Mohammad Babaeizadeh, Piotr Miłoś, Błażej Osinski, Roy H. Campbell, Konrad Czechowski, Dumitru Erhan, Chelsea Finn, Piotr Kozakowski, Sergey Levine, Afroz Mohiuddin, Ryan Sepassi, George Tucker, and Henryk Michalewski. Model-based reinforcement learning for atari. In *International Conference on Learning Representations*, 2020.
- Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. URL <http://jmlr.org/papers/v22/20-1364.html>.
- Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714*, 2024.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.
- Richard S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In *Proceedings of the Seventh International Conference on Machine Learning*, pages 216–224, 1990.
- Richard S. Sutton. Dyna, an integrated architecture for learning, planning, and reacting. *ACM SIGART Bulletin*, 2(4):160–163, 1991.

Richard S. Sutton. The grand challenge of predictive empirical abstract knowledge. In *Working Notes of the IJCAI-09 Workshop on Grand Challenges for Reasoning from Experiences*, 2009.

Richard S. Sutton. The bitter lesson. incompleteideas.net, 2019.

Richard S. Sutton, Joseph Modayil, Michael Delp, Thomas Degris, Patrick M. Pilarski, Adam White, and Doina Precup. Horde: A scalable real-time architecture for learning knowledge from unsupervised sensorimotor interaction. In *Proceedings of the 10th International Conference on Autonomous Agents and Multiagent Systems*, pages 761–768, 2011.

Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *Advances in Neural Information Processing Systems*, 2017.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

Weipu Zhang, Gang Wang, Jian Sun, Yetian Yuan, and Gao Huang. STORM: Efficient stochastic transformer based world models for reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.

Weipu Zhang, Adam Jelley, Trevor McInroe, Amos Storkey, and Gang Wang. Object-centric world models from few-shot annotations for sample-efficient reinforcement learning. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=qmEyJadwHA>.

A Exact Quantitative Scores

Table 2: Exact scores for Figure 2. Reproduced-agent and frozen-WM scores are reported as mean $\pm$ sample standard deviation over 20 real-ALE reset seeds with deterministic action selection. The original-paper column reports the Pong score reported by the corresponding world-model paper. Higher is better.

World model	Original paper	Reproduced agent	Frozen WM	Change
DreamerV3	−4.0	−5.45 $\pm$ 3.32	−20.90 $\pm$ 0.31	−15.45
DIAMOND	20.4	19.70 $\pm$ 2.43	−9.55 $\pm$ 12.58	−29.25
TWISTER	20.0	17.70 $\pm$ 1.26	−13.30 $\pm$ 7.84	−31.00
Simulus	19.9	20.75 $\pm$ 0.55	−11.60 $\pm$ 8.27	−32.35
STORM	11.0	18.70 $\pm$ 1.59	−21.00 $\pm$ 0.00	−39.70

Table 3: Exact Pong scores corresponding to Figure 4, reported as mean $\pm$ sample standard deviation over 20 real-ALE reset seeds with deterministic action selection. Higher is better.

World model	Reproduced-agent WM	Offline baseline WM	Offline CGSReg WM
DreamerV3	−20.95 $\pm$ 0.22	−21.00 $\pm$ 0.00	− 11.90 $\pm$ 5.66
DIAMOND	−19.65 $\pm$ 1.53	−13.90 $\pm$ 4.51	− 5.80 $\pm$ 6.63
TWISTER	−8.40 $\pm$ 6.41	−21.00 $\pm$ 0.00	− 1.90 $\pm$ 21.26
Simulus	−9.30 $\pm$ 9.57	−15.80 $\pm$ 6.57	− 4.10 $\pm$ 15.78
STORM	−21.00 $\pm$ 0.00	−21.00 $\pm$ 0.00	−21.00 $\pm$ 0.00

B CGSReg Details and Project Implementations

Section 3 defines CGSReg as a masked image-space loss. This section provides additional details about loss reduction, multiple concept masks, and project-specific implementations.

B.1 Loss Reduction

Image losses in visual world models use different reduction conventions. A mean-reduced loss averages over image elements, whereas a sum-reduced loss accumulates their errors. Because CGSReg uses a region-normalized MSE, we scale it to match the reduction convention of the original image loss.

For a single concept mask m , the region-normalized loss is

$$\ell(m) = \frac{\sum_p m_p (x_p - \hat{x}_p)^2}{\sum_p m_p}. \quad (4)$$

If the original image loss uses mean reduction, we directly use

$$L_{\text{CGSReg}} = \ell(m). \quad (5)$$

If the original image loss uses sum reduction, we rescale the region-normalized loss by the number of elements in the full image:

$$L_{\text{CGSReg}} = |\Omega| \ell(m), \quad (6)$$

where Ω denotes the full image support. For an image with shape (H, W, C) ,

$$|\Omega| = HWC. \quad (7)$$

This scaling makes the magnitude of CGSReg comparable to the original image loss under different reduction conventions.

B.2 Multiple Concept Masks

CGSReg can be applied to multiple concept regions. Let $\mathcal{M}_{\text{enabled}}$ denote the set of enabled binary masks. We first compute the unscaled aggregate loss

$$L'_{\text{CGSReg}} = \sum_{m \in \mathcal{M}_{\text{enabled}}} \ell(m). \quad (8)$$

Each mask is normalized by its own region size before aggregation. Different concept regions therefore contribute according to their reconstruction errors rather than their spatial sizes.

The reduction matching described above is then applied to the aggregate loss:

$$L_{\text{CGSReg}} = \begin{cases} L'_{\text{CGSReg}}, & \text{for mean reduction,} \\ |\Omega| L'_{\text{CGSReg}}, & \text{for sum reduction.} \end{cases} \quad (9)$$

In the main experiments, the ball mask is the only active concept region.

B.3 Implementation in Each Project

CGSReg is defined on image-space losses and does not require architectural changes. For each project, we apply it to the image prediction or reconstruction objective of the corresponding visual learning module. Table 4 summarizes the implementation details.

C SAM2 Segmentation Pipeline

We augment the 100K-step Atari Pong replay dataset with concept masks generated by a guided SAM2 [Ravi et al., 2024] video-segmentation pipeline. Each replay frame is paired with its corresponding object masks, which are stored as additional dataset fields and used as spatial supervision during world-model training.

Table 4: CGSReg implementation in each world-model project.

Project	Where CGSReg is applied	Implementation note
DreamerV3	Decoded-image reconstruction from RSSM latents	Uses sum-reduction scaling to match the original image loss.
DIAMOND	Denoising image loss	Uses the region-normalized masked loss directly.
TWISTER	Decoded-image reconstruction from latent states	Uses the region-normalized masked loss directly.
Simulus	Tokenizer image reconstruction	Applied only to tokenizer reconstruction, not token dynamics.
STORM	Decoded-image reconstruction from predicted latents	Uses the region-normalized masked loss directly.

Directly applying SAM2 to every replay trajectory would require manually providing object prompts for each trajectory, which is impractical for a large replay dataset. Instead, we create a small set of representative guiding frames. Each guiding frame is an auxiliary frame with manually specified object point prompts. These guiding frames are inserted into replay trajectories and provide reusable prompts for SAM2 video propagation. The same object annotations can therefore be transferred to many replay trajectories without additional manual labeling.

Because applying SAM2 to long trajectories can exceed GPU memory, we process each replay trajectory in smaller temporal chunks. For each chunk, we insert a set of corresponding guiding frames, run SAM2 video segmentation, remove the inserted frames from the output, and align the remaining masks with the original replay frames.

For Atari Pong, we generate three object masks:

- mask1: ball;
- mask2: opponent paddle;
- mask3: player paddle.

The final replay dataset stores the original observations together with their aligned concept masks. The main CGSReg experiments use only mask1.

Algorithm 2 Guided SAM2 segmentation with reusable guiding frames

- 1: **Input:** replay trajectories $\{V_i\}_{i=1}^N$, guiding frames $\{(g_j, P_j)\}_{j=1}^K$, and chunk length L
 - 2: Initialize empty mask dataset D_{mask}
 - 3: **for** each replay trajectory $V_i = (x_1, \dots, x_T)$ **do**
 - 4: **for** each temporal chunk C of V_i with length L **do**
 - 5: Select guiding frames and prompts $\{(g_j, P_j)\}_{j \in \mathcal{J}}$ for the chunk
 - 6: Insert guiding frames into C to obtain augmented chunk C^+
 - 7: Run SAM2 segmentation: $Y^+ = \text{SAM2}(C^+, \{P_j\}_{j \in \mathcal{J}})$
 - 8: Remove masks corresponding to inserted guiding frames
 - 9: Align the remaining masks with frames in C
 - 10: Store the aligned masks in D_{mask}
 - 11: **end for**
 - 12: **end for**
 - 13: Return the replay dataset augmented with concept masks
-

D Offline World-Model Training

The matched offline baseline and CGSReg checkpoints are trained on the same fixed replay dataset containing 100k Atari Pong steps collected from our DIAMOND reproduction. Only world-model components are optimized; policy, actor, and critic updates are disabled.

Because the original projects report training budgets in different units, we align the offline runs by the number of gradient updates applied to world-model components. Table 5 summarizes the matched budgets and updated components.

Table 5: Training budgets used for offline world-model training. Update counts refer only to world-model optimization steps.

Project	Offline world-model updates	Updated components
DreamerV3	25k	RSSM and image decoder
DIAMOND	409.6k	Diffusion world model
TWISTER	100k	Latent dynamics and image decoder
Simulus	119k tokenizer updates + 115k dynamics updates	Image tokenizer and token dynamics
STORM	100k	Latent dynamics and image decoder

For DreamerV3, 100k reproduced environment steps correspond to 25k world-model updates. For DIAMOND, 1000 epochs correspond to 409.6k denoiser updates. For TWISTER, Simulus, and STORM, we directly count world-model updates while excluding policy and critic updates.

E Unified Pixel-Space Zero-Shot MBRL Protocol

This section describes the unified zero-shot protocol used in subsection 4.3. For each frozen world-model checkpoint, we train a new policy entirely inside the frozen model and evaluate the resulting policy in the real ALE Pong environment. The world-model parameters remain fixed, and no additional real-environment data are collected.

At the beginning of each episode, the frozen world model receives an initial context sampled from the fixed replay dataset. It then generates future observations, rewards, and terminal signals conditioned on the policy actions. The policy receives only pixel observations: $64 \times 64 \times 3$ RGB images, without access to latent representations or internal environment states.

We use the same policy-learning algorithm and hyperparameters for all world models. The policy is a discrete-action on-policy actor-critic agent with an image encoder, a categorical policy head, and a value head. Each update collects 15 rollout steps from 64 parallel world-model environments, followed by one optimization step. We train for 20k updates, corresponding to

$$64 \times 15 \times 20000 = 19.2\text{M}$$

world-model environment steps.

The rollout horizon inside each frozen world model is 512. Reaching this horizon is treated as truncation rather than termination. Rewards follow the Atari Pong convention $-1, 0, +1$. For models that produce continuous rewards, we apply a threshold of 0.1 before policy learning. After training, we evaluate the learned policy over 20 real-ALE reset seeds with deterministic action selection. Pong returns lie in $[-21, 21]$, where higher scores indicate better performance.

F Ablations

The main experiments select one λ_{CGSReg} for each project and use a fixed replay dataset. This section studies two factors that may affect the effectiveness of CGSReg: the regularization strength and the offline replay dataset. All ablation experiments use the unified pixel-space zero-shot MBRL protocol from subsection 4.3.

We sweep $\lambda_{\text{CGSReg}} \in \{0, 0.01, 0.1, 1.0\}$ on two fixed replay datasets. The main experiments use the dataset collected from our DIAMOND reproduction. The second dataset is collected from our STORM reproduction [Zhang et al., 2023].

The ablation results show that CGSReg improves DreamerV3, DIAMOND, and TWISTER on both replay datasets, although the optimal λ_{CGSReg} depends on the model and dataset.

G Representative World-Model Projects

This section provides an overview of the five representative visual world-model agents evaluated in this work. These projects explore different architectural designs and prediction paradigms, including

Table 6: Regularization-strength and replay-dataset ablations under the unified pixel-space zero-shot MBRL protocol. Each entry reports mean $\pm$ sample standard deviation over 20 real-ALE reset seeds with deterministic action selection. Higher is better.

Dataset	World model	$\lambda_{\text{CGSReg}} = 0$	$\lambda_{\text{CGSReg}} = 0.01$	$\lambda_{\text{CGSReg}} = 0.1$	$\lambda_{\text{CGSReg}} = 1.0$
DIAMOND exp-repro	DreamerV3	-21.00 ± 0.00	-21.00 ± 0.00	-11.90 ± 5.66	-21.00 ± 0.00
	DIAMOND	-13.90 ± 4.51	-5.80 ± 6.63	-20.90 ± 0.31	-20.80 ± 0.41
	TWISTER	-21.00 ± 0.00	-21.00 ± 0.00	-16.10 ± 7.27	-1.90 ± 21.26
	Simulus	-15.80 ± 6.57	-5.10 ± 5.68	-4.10 ± 15.78	-20.75 ± 0.44
	STORM	-21.00 ± 0.00	-21.00 ± 0.00	-21.00 ± 0.00	-21.00 ± 0.00
STORM exp-repro	DreamerV3	-18.55 ± 2.46	-15.15 ± 9.43	-20.20 ± 0.70	-21.00 ± 0.00
	DIAMOND	-20.30 ± 0.47	-20.25 ± 1.33	-4.60 ± 15.26	-20.80 ± 0.41
	TWISTER	-21.00 ± 0.00	-17.20 ± 3.40	-20.80 ± 0.41	-21.00 ± 0.00
	Simulus	0.95 ± 8.13	2.80 ± 0.89	-17.65 ± 2.78	-19.15 ± 0.99
	STORM	-21.00 ± 0.00	-21.00 ± 0.00	-21.00 ± 0.00	-21.00 ± 0.00

recurrent latent dynamics models, pixel-space diffusion models [Ho et al., 2020], Transformer-based latent sequence models [Vaswani et al., 2017], and discrete token-based predictive models.

DreamerV3, DIAMOND, TWISTER, and STORM are trained end to end, jointly optimizing visual representation learning and future dynamics prediction. Simulus instead adopts a two-stage framework that first learns a visual tokenizer and then trains a separate token-dynamics model.

The evaluated agents also differ in their prediction spaces. DIAMOND directly models future observations in pixel space using an action-conditioned diffusion model. DreamerV3, TWISTER, and STORM predict future continuous latent states. Simulus performs next-token prediction in a discrete visual token space using a RetNet-based autoregressive model [Sun et al., 2023]. Despite these differences, all five agents include visual learning objectives that encourage consistency with future observations.

Table 7: Overview of representative visual world models evaluated in this work.

Project	Prediction space	Backbone	Main design principle
DreamerV3	Latent state	RSSM	Recurrent latent imagination
DIAMOND	Pixel space	Diffusion model	Action-conditioned image generation
TWISTER	Latent state	Transformer	Contrastive latent dynamics learning
Simulus	Discrete tokens	RetNet	Token-level autoregressive prediction
STORM	Latent state	Transformer	Stochastic sequence modeling

G.1 DreamerV3

DreamerV3 [Hafner et al., 2023] is a representative latent world-model-based reinforcement learning framework. It uses a recurrent state-space model (RSSM) to model environment dynamics in a compact latent space. The RSSM state contains a deterministic recurrent state and a stochastic latent state represented by discrete categorical variables. Conditioned on previous latent states and actions, the RSSM predicts future latent states, while decoder networks reconstruct observations and predict auxiliary signals such as rewards and continuation probabilities.

DreamerV3 performs reinforcement learning inside the learned world model through latent imagination. Because dynamics prediction occurs in latent space, policy optimization uses latent-state trajectories rather than pixel observations. DreamerV3 stabilizes latent world-model training through design choices including discrete categorical latent variables, KL balancing, and symlog-based prediction targets.

These design choices enable DreamerV3 to perform strongly across diverse reinforcement learning benchmarks. It is competitive on Atari 100K, scales to the full Atari benchmark, and applies to continuous-control tasks such as the DeepMind Control Suite and long-horizon environments such as Minecraft.

G.2 DIAMOND

DIAMOND [Alonso et al., 2024] is a pixel-space visual model-based reinforcement learning agent that uses a diffusion model for action-conditioned future prediction. Given previous observations and

actions, DIAMOND predicts future frames by iteratively denoising noisy images. It therefore models future pixels directly rather than learning an intermediate latent dynamics model. This design enables high-fidelity visual rollouts, strong Atari 100K results, and an interactive neural game engine trained from Counter-Strike: Global Offensive gameplay.

G.3 TWISTER

TWISTER [Burchi and Timofte, 2025] adopts a latent-space world-model architecture similar to STORM, using a Transformer-based dynamics model to predict action-conditioned future latent states. It follows the Dreamer-style latent-state modeling paradigm while replacing recurrent dynamics with Transformer sequence modeling. Its main contribution is the action-conditioned contrastive predictive coding (AC-CPC) objective, which trains the model to predict and distinguish action-conditioned future latent states and preserve information useful for long-horizon prediction.

G.4 Simulus

Simulus [Cohen et al., 2025] is the successor to REM [Cohen et al., 2024] and adopts a token-based visual world-modeling paradigm. In REM, the visual world model is trained in two stages: a VQ-VAE tokenizer [van den Oord et al., 2017] first represents image observations as discrete visual tokens, after which a RetNet-based autoregressive dynamics model [Sun et al., 2023] learns next-token prediction conditioned on actions. Simulus extends this framework with multi-modal tokenization and several training improvements. Unlike the other evaluated agents, it models future visual evolution through next-token prediction in a discrete token space.

G.5 STORM

STORM [Zhang et al., 2023] follows the Dreamer-style latent world-model design, where each latent state contains a deterministic representation and a stochastic latent variable. Instead of the recurrent state-space model used by DreamerV3, STORM employs a Transformer-based sequence model to predict action-conditioned future latent trajectories. Its successor, OC-STORM [Zhang et al., 2026], further incorporates object-centric representations into the framework.

H Diagnostic Study of World-Model Freezing in Dyna-Style Training

The strong RL performance of Dyna-style agents contrasts with the poor standalone performance of their frozen world models. Although these agents can achieve high returns during joint training, the corresponding frozen world models often produce incorrect closed-loop rollouts and support weak pixel-space zero-shot MBRL performance.

One possible explanation is that Dyna-style agents benefit from continual world-model updating. During Dyna-style training, the policy and world model are optimized together, while newly collected real-environment data continually update the world model. The policy may therefore benefit from a model that is useful near its current data distribution without requiring the model to become a strong standalone simulator after freezing.

This hypothesis predicts that freezing the world model during Dyna-style training should reduce final RL performance, especially when freezing occurs earlier. To test this hypothesis, we freeze the world model at different training stages and continue the remaining training process. For each project, we follow its native Dyna-style training pipeline. After freezing, real-environment interaction, data collection, policy learning, and evaluation continue, while only world-model updates are disabled.

Let T denote the nominal reproduction training budget. We compare four settings: no freeze, freeze at $0.5T$, freeze at $0.75T$, and freeze at $1.0T$. All runs continue until $1.5T$. This diagnostic follows each project’s native training and evaluation protocol and is therefore separate from the unified pixel-space zero-shot MBRL protocol in subsection 4.3.

Figure 5 shows the performance trajectories after freezing world-model updates. Table 8 reports the final real-environment returns.

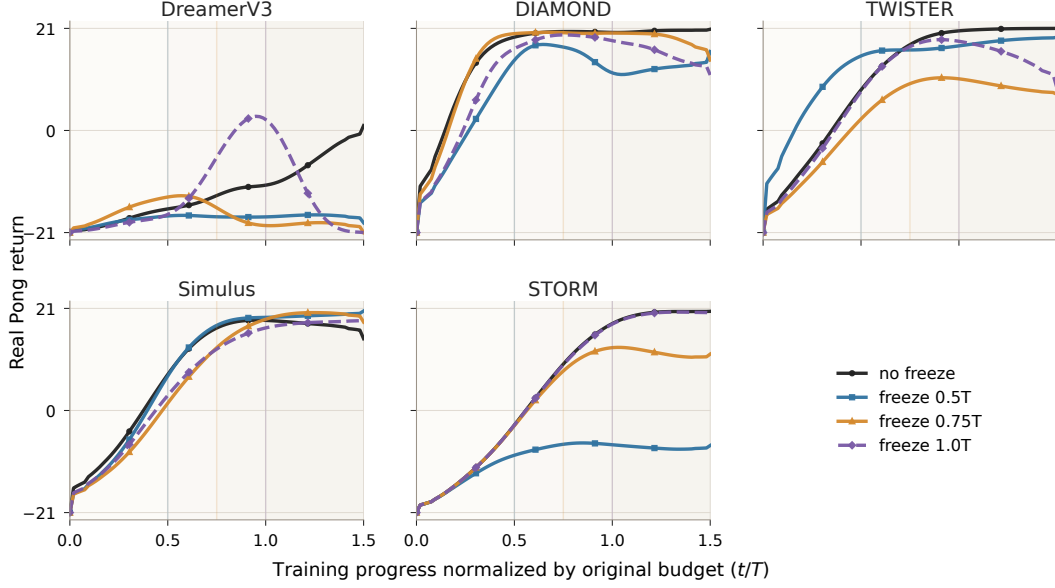


Figure 5: Scores after freezing world-model updates during Dyna-style training. The horizontal axis is normalized training progress t/T , where T is each project’s original reproduction budget. All runs continue to $1.5T$. For visual comparison across projects with different evaluation frequencies, the score curves are smoothed and resampled to a shared training-progress grid. Final unsmoothed scores are reported in Table 8.

Table 8: Final real-environment returns after freezing world-model updates during Dyna-style training. All runs continue to $1.5T$, so the no-freeze scores are not identical to the reproduced-agent scores in Table 2.

World model	No freeze	Freeze 0.5T	Freeze 0.75T	Freeze 1.0T
DreamerV3	1.00	−19.00	−21.00	−21.00
DIAMOND	20.82	16.23	14.49	11.41
TWISTER	21.00	19.16	7.44	8.80
Stimulus	14.77	20.47	18.18	18.92
STORM	20.40	−7.20	11.60	20.00

The results provide partial support for the hypothesis. DreamerV3 and DIAMOND show lower post-freeze trajectories than their no-freeze controls, suggesting that continual world-model updating contributes to their Dyna-style performance. TWISTER shows a weaker effect, while Stimulus does not degrade and STORM exhibits non-monotonic behavior. Continual world-model updating can therefore be important for some architectures, but it does not fully explain why frozen world models remain weak standalone policy-learning simulators.

The cause of the frozen-world-model gap remains unclear. Other factors, including robustness to policy shifts, long-horizon consistency, and action-conditioned prediction, may also contribute and remain important directions for future work.